# 🧪 Unit Testing with JUnit in Spring Boot

## Beginner ➜ Pro Guide

*“Good developers test their code. Great developers design systems that are easy to test.”*

When teams skip testing → bugs reach production.
When teams test properly → releases become boring (and boring is good).

JUnit + Spring Boot gives us a **simple, powerful testing ecosystem**.

Let’s walk through it step-by-step.

---

# 🟢 STEP 1 — Why Unit Testing Matters

Imagine this production scenario:

```
PaymentService
  ↓
Apply discount
  ↓
Charge card
```

One developer changes this line:

```
price = price * 2;
```

Suddenly customers are charged **twice**.

Without tests → production disaster.

With tests → build fails instantly.

Unit tests are your **silent bodyguards**.

---

# 🟢 STEP 2 — Project Setup

Spring Boot makes testing easy.

### Gradle dependency

```
dependencies {
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
}
```

This includes:

| Library | Purpose |
|---|---|
| JUnit 5 | Testing framework |
| Mockito | Mock dependencies |
| AssertJ / Assertions | Test validation |
| Spring Test | Integration testing |

---

# 🟢 STEP 3 — Your First JUnit Test

## Calculator Example

### Class to test

```
public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }
}
```

### Test

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```
class CalculatorTest {

    @Test
    void testAdd() {
        Calculator calculator = new Calculator();
        assertEquals(5, calculator.add(2, 3));
    }
}
```

### What happens

```
Test Runner
  ↓
Execute testAdd()
  ↓
Compare expected vs actual
```

If values differ → test fails.

---

# 🟡 STEP 4 — Testing Spring Services

Real apps use Spring Beans.

Example service:

```
@Service
public class UserService {

    public String greet(String name) {
        return "Hello " + name;
    }
}
```

### Test with Spring Context

```
@SpringBootTest
class UserServiceTest {

    @Autowired
```

```
    UserService userService;

    @Test
    void testGreet() {
        assertEquals("Hello John", userService.greet("John"));
    }
}
```

### Important

`@SpringBootTest` loads the entire Spring context.

Think of it like:

Starting a car just to test the radio.

Useful, but slower.

---

# 🟡 STEP 5 — Real Unit Testing with Mockito

In professional environments we avoid starting Spring when possible.

Example Service:

```
@Service
public class ProductService {

    @Autowired
    ProductRepository repo;

    public Product getProduct(Long id) {
        return repo.findById(id).orElseThrow();
    }
}
```

### Unit Test

```
@ExtendWith(MockitoExtension.class)
class ProductServiceTest {

    @Mock
    ProductRepository repo;

    @InjectMocks
    ProductService productService;

    @Test
    void testGetProduct() {

        Product p = new Product(1L, "Laptop");

        when(repo.findById(1L))
            .thenReturn(Optional.of(p));

        Product result = productService.getProduct(1L);

        assertEquals("Laptop", result.getName());
    }
}
```

### Why this is better

| Feature | Benefit |
|---|---|
| Mock repository | No database |
| Fast execution | Milliseconds |
| True unit testing | Service isolated |

---

# 🔵 STEP 6 — Testing REST Controllers

Controllers expose APIs.

We test them with **MockMvc**.

---

## Example Controller

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    ProductService productService;

    @GetMapping("/{id}")
    public Product getProduct(@PathVariable Long id) {
        return productService.getProduct(id);
    }
}
```

---

## Controller Test

```
@WebMvcTest(ProductController.class)
class ProductControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private ProductService productService;

    @Test
    void testGetProduct() throws Exception {

        Product p = new Product(1L, "Phone");

        when(productService.getProduct(1L)).thenReturn(p);

        mockMvc.perform(get("/api/products/1"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.name").value("Phone"));
    }
}
```

### What MockMvc Does

Fake HTTP Request

↓
Controller
↓
Service (Mock)
↓
JSON Response

No real server needed.

---

# 🔴 STEP 7 — Testing Exceptions

Real systems fail.

Your tests should verify **failure behavior**.

```
@Test
void testProductNotFound() {

    when(repo.findById(99L)).thenReturn(Optional.empty());

    assertThrows(
        NoSuchElementException.class,
        () -> productService.getProduct(99L)
    );
}
```

---

# 🟣 STEP 8 — Edge Case Testing

Professionals test **weird inputs**.

Examples:

| Case | Why |
|---|---|
| Null inputs | Prevent crashes |
| Large numbers | Prevent overflow |

| | |
|---|---|
| Empty lists | Prevent logic errors |
| Negative values | Business logic validation |

Example:

```
@Test
void testAddEdgeCase() {

    Calculator c = new Calculator();

    assertEquals(Integer.MAX_VALUE,
                 c.add(Integer.MAX_VALUE, 0));
}
```

---

# 🟤 STEP 9 — Testing Database Layer

Use **in-memory DB**.

Annotation:

```
@DataJpaTest
```

---

### Repository Test

```
@DataJpaTest
class ProductRepositoryTest {

    @Autowired
    ProductRepository repo;

    @Test
    void testSaveProduct() {

        Product p = new Product(null, "Tablet");

        Product saved = repo.save(p);

        assertNotNull(saved.getId());
```

```
    }
}
```

Spring Boot automatically configures:

H2 Database
Hibernate
JPA

Fast and isolated.

---

# ⚙️ STEP 10 — Test Naming Best Practices

Bad test name:

test1()

Professional test name:

testCalculateTax_whenStateIsCA_shouldReturnCorrectAmount()

Pattern:

methodName_whenCondition_shouldExpectedResult

---

# 🧠 STEP 11 — Maintainable Test Suites

Lessons from large production systems:

✔ Test **one scenario per method**
✔ Avoid unnecessary Spring context
✔ Use **constructor injection**
✔ Avoid magic numbers
✔ Keep tests readable

Bad test:

assertEquals(4, service.calculate(2,2));

Better:

assertEquals(EXPECTED_TOTAL, service.calculate(PRICE, TAX));

---

# 📊 STEP 12 — Code Coverage (Reality Check)

Tools like:

- JaCoCo
- SonarQube

Measure coverage.

Example:

Coverage = 85%

But remember:

100% coverage does not mean 100% correct.

Focus on **meaningful scenarios**, not metrics.

---

# 🏗️ STEP 13 — Full Stack Test Strategy

Example API:

GET /books/50

Architecture:

Controller
↓
BookService
↓
BookRepository
↓
Database

Testing layers:

| Layer | Test Type |
| --- | --- |
| Controller | MockMvc |
| Service | Mockito |
| Repository | DataJpaTest |
| Exception | Unit Test |

Each layer tested **independently**.

---

# 🏆 Professional Testing Pyramid

E2E Tests
(Few, slow)

Integration Tests
(Some, medium)

Unit Tests
(Many, very fast)

Goal:

70% Unit Tests
20% Integration
10% E2E

---

# 💡 Final Wisdom:

The difference between **junior developers and elite engineers**:

| Junior | Expert |
|---|---|
| Writes code | Designs testable code |
| Tests manually | Automates tests |
| Fixes bugs | Prevents bugs |

Testing isn't about catching mistakes.

It’s about **engineering confidence**.

When your test suite is strong:

Deployments become boring.
Boring deployments are successful deployments.

---